UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

WEB TECHNOLOGIES – LECTURE 21

FROM WORKING APIs TO PRODUCTION-READY APIs

Luigi Libero Lucio Starace, PhD

luigiliberolucio.starace@unina.it

https://luistar.github.io; https://squids.unina.it

https://www.docenti.unina.it/luigiliberolucio.starace

TODAY'S GOAL

- A backend is not finished as soon as the endpoints work
- A backend becomes mature when it can evolve, survive clients, resist abuse, and be operated safely.
- Some questions we should be able to answer:
  - How do we document an API?
  - How do we change the API without breaking existing clients?
  - How do we protect resources from unauthorized access?
  - How do we limit abuse and excessive traffic?
  - How do we observe what is happening in production?
  - How do we safely retire old functionality?

OPENAPI: DESCRIBING A REST API

- **OpenAPI** (formerly Swagger) is an open, formal standard to describe HTTP APIs
- Why?
  - **Standardization**: ensures consistent documentation practices
  - **Documentation**: can be used to automatically generate comprehensive docs
  - **Code generation**: Allows for automatic generation of client libraries and server stubs
  - **Machine and human-readable:** enables automated discovery and more
  - **Most broadly adopted industry standard**



THE OPENAPI SPECIFICATION

- **OpenAPI Descriptions** are written as structured text documents
- Each document represents a JSON object, in JSON or YAML format
- These specification files describe version of the OpenAPI specification (`openapi`), the API (`info`) and the endpoints (`paths`)

```yaml
# YAML format
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {} # No endpoints defined
```

```json
// JSON format
{
  "openapi": "3.1.0",
  "info": {
    "title": "A minimal specification",
    "version": "0.0.1"
  },
  "paths": {} // No endpoints defined
}
```

THE OPENAPI SPECIFICATION: PATHS

- The API Endpoints are called **Paths** in the OpenAPI specification
- **Path** objects allow to specify a sequence of endpoints
  - For each endpoint, its supported methods
  - For each method,
    - Expected input parameters (if any) (e.g.: path parameters, headers, etc…)
    - Request body (if any)
    - Possible responses
    - And more!

THE OPENAPI SPECIFICATION: PATHS

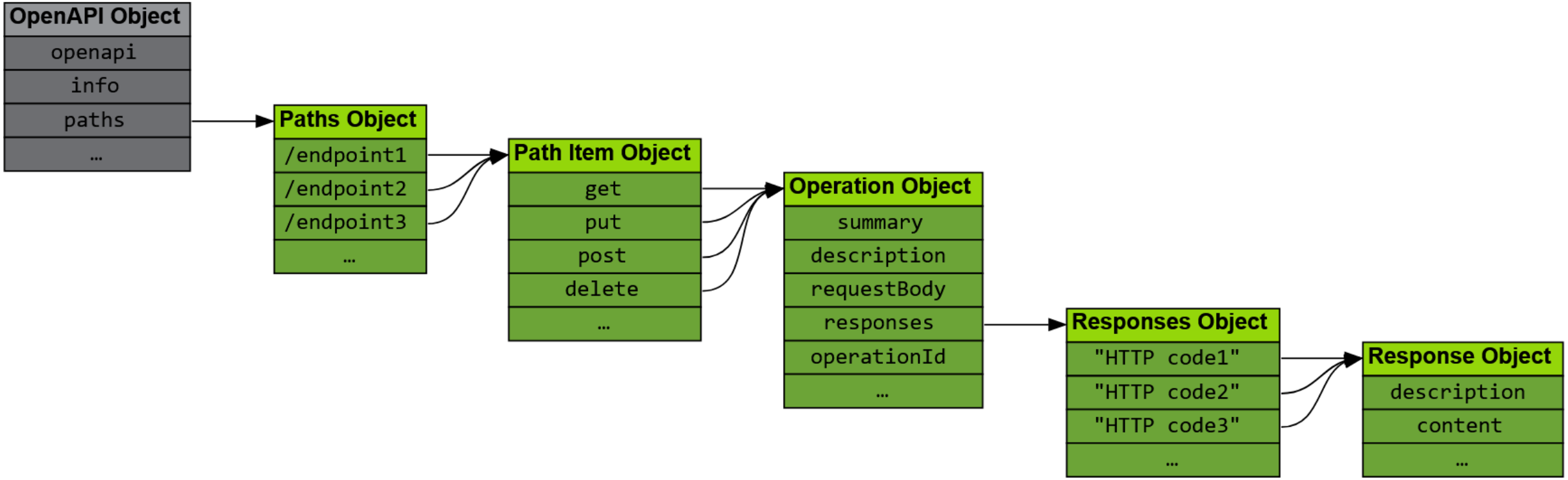


Image from https://learn.openapis.org/specification/paths

PATH EXAMPLE

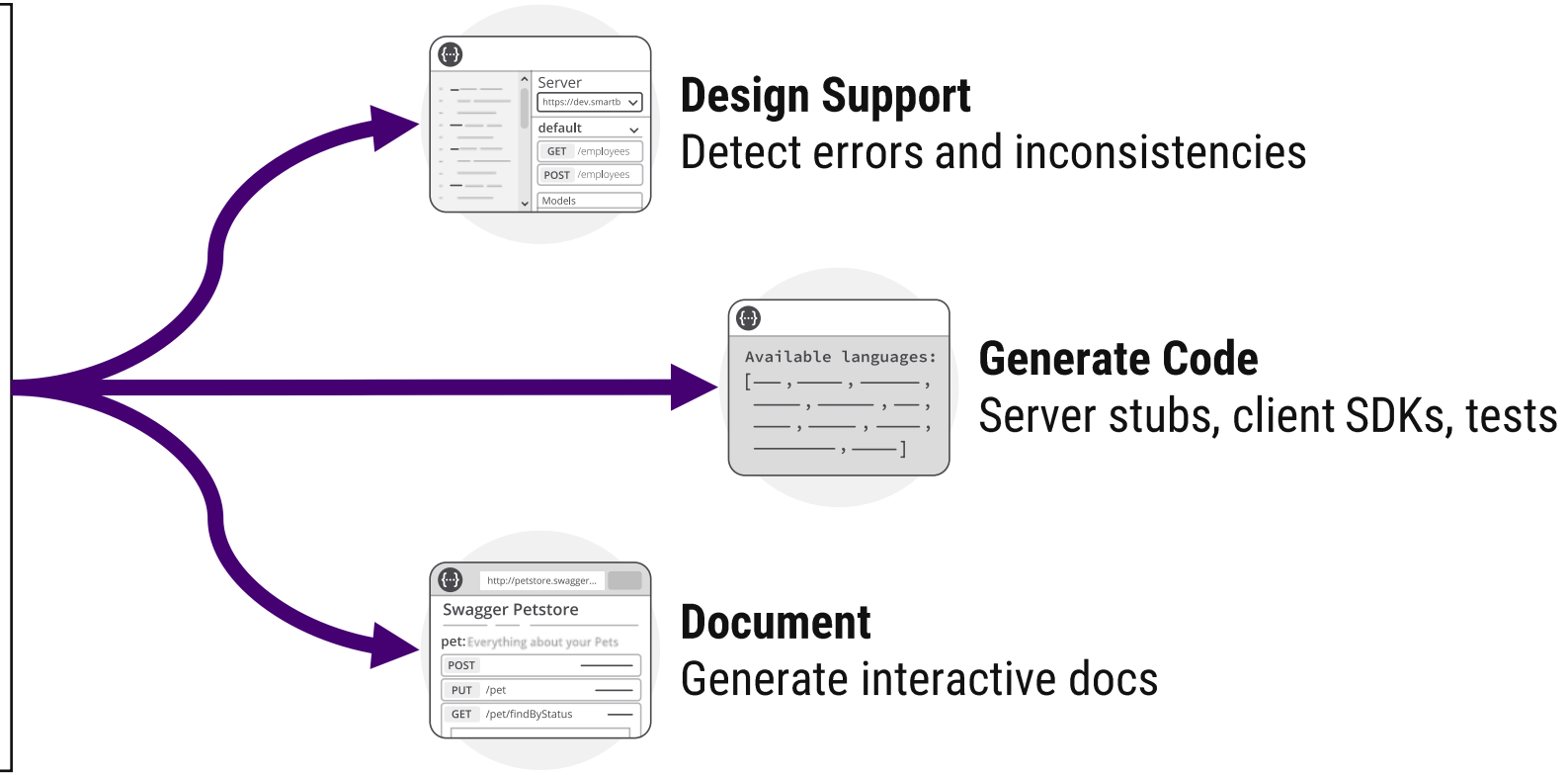
- /auth endpoint, POST method
- Expects Request Body containing a JSON object with type {usr: string, pwd: string}
- Returns 200 OK on success, 401 Unauthorized when credentials are invalid

```yaml
/auth:
  post:
    description: Authenticate user
    produces:
      - application/json
    requestBody:
      description: user to authenticate
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              usr:
                type: string
                example: Kyle
              pwd:
                type: string
                example: p4ssw0rd
    responses:
      200:
        description: User authenticated
      401:
        description: Invalid credentials
```

WORKING WITH OPENAPI SPECIFICATIONS

- Writing an OpenAPI specification for an API is quite a lot of work
- What can we do with an OpenAPI specification?

```yaml
openapi: 3.1.0
info:
  title: My API
  version: 1.0.0
paths:
  /auth:
    post:
      # ...
  /todos:
    get:
      # ...
    post:
      # ...
```



Design Support
Detect errors and inconsistencies

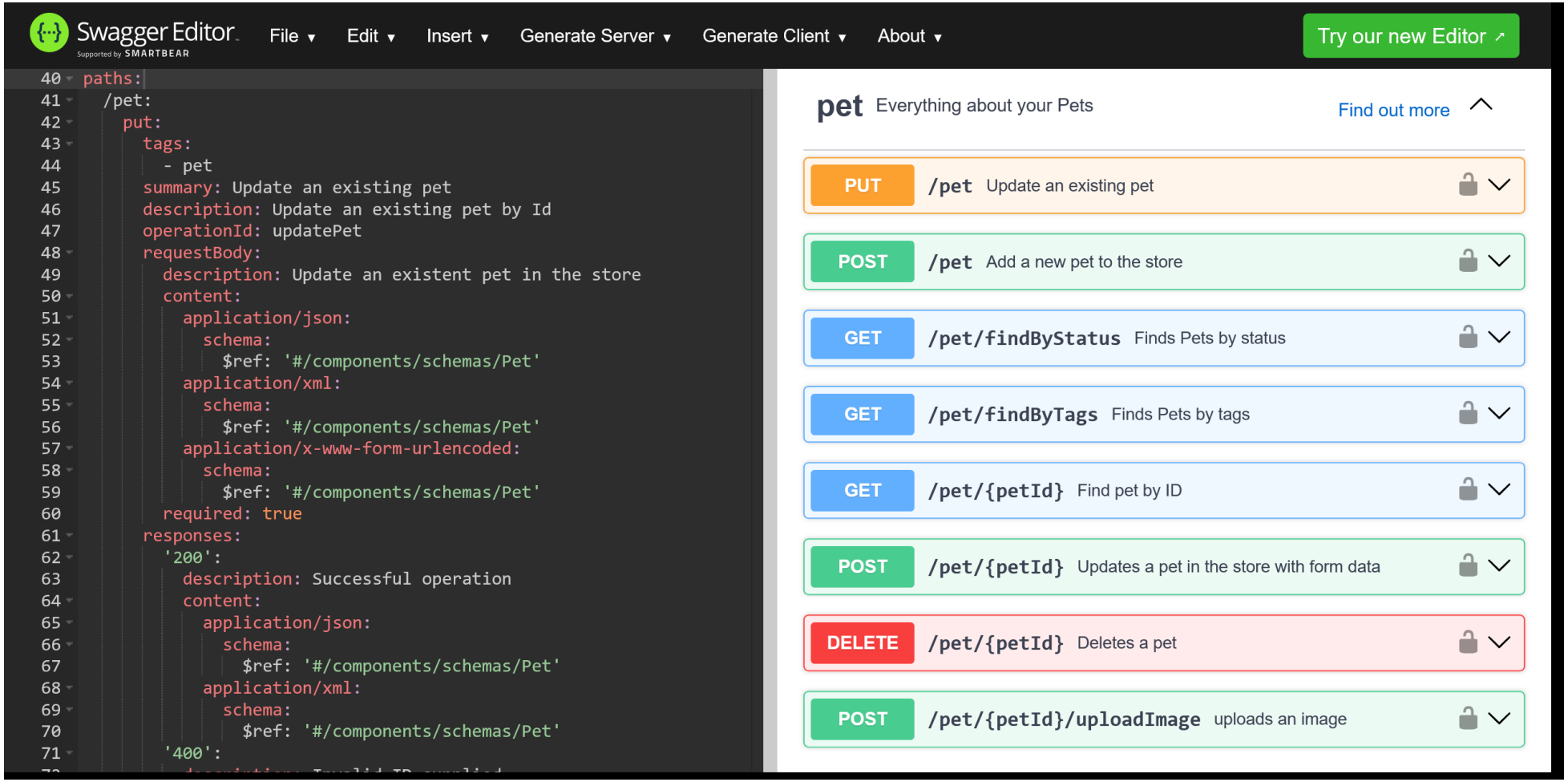
Generate Code
Server stubs, client SDKs, tests

Document
Generate interactive docs

OPENAPI: OPEN SOURCE TOOLS

- OpenAPI is supported by a number of mature, open-source tools
- Some widely used open-source tools are reported as follows:
  - **Swagger Editor**: https://swagger.io/tools/swagger-editor/
  - **Swagger Codegen**: https://swagger.io/tools/swagger-codegen/
  - **Swagger UI**: https://swagger.io/tools/swagger-ui/
- A free alternative is https://openapi-generator.tech/

OPENAPI: SWAGGER EDITOR



Specification

Documentation (Swagger UI)

OPENAPI: CODEGEN

- From the editor toolbar, we can also generate Server and Client code
- Server-side code generation can obviously only provide stubs
  - Supported languages/frameworks include: ASP.NET, Go, Java, JaxRS, Micronaut, Kotlin, Node.js, Python (Flask), Scala, Spring.
- Client-side code generation is a very handy way to make SDKs for our APIs available in tens of different languages
  - Supported languages include: C#, Dart, Go, Java, JavaScript, Kotlin, PHP, Python, R, Ruby, Scala, Swift, TypeScript

OPENAPI-DRIVEN DEVELOPMENT

- Often, developers start working on an API by defining its specification before writing any actual code
- This approach is also referred to as **OpenAPI-driven development**
- This way, they can exploit code generation capabilities to the fullest
- The approach also promotes **independence** between the different teams involved in a project (front-end, back-end, QA). The API definition keeps all these stakeholders aligned

API EVOLUTION



THE API IS A CONTRACT

- When we expose an API, we create a contract.
- The contract includes:
  - URL paths and HTTP methods
  - Request body format
  - Response body format
  - Status codes
  - Authentication requirements and Authorization rules
  - Error format
  - Performance expectations
- Even if we do not write the contract down, clients will depend on it.

API EVOLUTION: WHY IT HAPPENS

- APIs, like any other meaningful software, will evolve over time
- Some change causes may be:
  - Business rule change
  - Frontend requirements change
    - Mobile apps need new fields
  - Old data model becomes insufficient
  - Performance issues appear
  - Security requirements change
  - Integrations with third parties evolve

SAFE VS BREAKING CHANGES

- Some changes are «**safe**» with respect to the API contract
  - These change do not affect existing clients
- … while others are not safe
  - These changes can cause existing clients to stop working with the new API
  - They are referred to as (potentially) **breaking changes**

SAFE VS BREAKING CHANGES: EXAMPLES

USUALLY SAFE

- Add a new endpoint
- Add a new optional req. field
- Add a new optional res. field
- Add a new filter param. with a default
- Improve internal implementation (e.g.: refactoring) without altering behaviour

POTENTIALLY BREAKING

- Rename a field
- Remove a field
- Change a field type
- Change status codes
- Change error response format
- Change semantic behaviour of existing endpoint
- Change AuthN/Z requirements

SAFE CHANGE: EXAMPLE

- Adding a new field to a response is typically safe
- Clients should ignore unknown fields

```
{
  "id": 1,
  "title": "Prepare lecture",
  "completed": false
}
```



```
{
  "id": 1,
  "title": "Prepare lecture",
  "completed": false,
  "createdAt": "2026-05-16T09:30:00Z"
}
```

BREAKING CHANGE: EXAMPLE

- New requirement: we need to track task status with finer granularity
  - «TODO», «IN-PROGRESS», «DONE»

```
{
  "id": 1,
  "title": "Prepare lecture",
  "completed": false
}
```



```
{
  "taskId": 1,
  "name": "Prepare lecture",
  "status": "IN-PROGRESS"
}
```

EVOLUTIONARY STRATEGIES

- We cannot restrict our API to only safe changes
- At some points, breaking changes may be necessary
- When retro-compatibility is broken, we need an **evolutionary strategy**



API VERSIONING: BASIC IDEA

- When we make breaking changes, we may want to keep both older and newer versions of our API up (at least for some time)
- We should allow clients to use different versions of our API
- Common strategies include:
  - **URI Versioning**
    - `/api/v1/tasks`
    - `/api/v2/tasks`
  - Allowing clients to specify the version they want using **request headers**
    - `X-API-Version: 2`
    - `Accept: application/vnd.academy.tasks.v2+json`

URI VERSIONING

- `GET /api/v1/tasks/1`
- `GET /api/v2/tasks/1`
- Pros:
  - Easy to understand and test
  - Easy to document
  - Easy to route in most frameworks
- Disadvantages
  - Can create some duplications in controllers/routers

DJANGO NINJA EXAMPLE

```python
# api_v1.py
from ninja import NinjaAPI

api = NinjaAPI(version="1.0.0")

@api.get("/tasks/{task_id}")
def get_task(request, task_id: int):
    return {
        "id": task_id,
        "title": "Prepare lecture",
        "completed": False,
    }
```

```python
# api_v2.py
from ninja import NinjaAPI

api = NinjaAPI(version="2.0.0")

@api.get("/tasks/{task_id}")
def get_task(request, task_id: int):
    return {
        "id": task_id,
        "title": "Prepare lecture",
        "status": "OPEN",
    }
```

```python
from django.contrib import admin
from django.urls import path
from tasks.api_v1 import api as api_v1
from tasks.api_v2 import api as api_v2

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/v1/", api_v1.urls),
    path("api/v2/", api_v2.urls)
]
```

Docs will be generated in separed directories:

- http://localhost:8000/api/v1/docs
- http://localhost:8000/api/v2/docs

SPRING BOOT EXAMPLE

```java
@RestController
@RequestMapping("/api/v1/tasks")
public class TaskControllerV1 {

    @GetMapping("/{id}")
    public TaskResponseV1 getTask(@PathVariable Long id) {
        return new TaskResponseV1(
                id,
                "Prepare lecture",
                false
        );
    }
}

public record TaskResponseV1(
        Long id,
        String title,
        boolean completed
) {}
```

```java
@RestController
@RequestMapping("/api/v2/tasks")
public class TaskControllerV2 {

    @GetMapping("/{id}")
    public TaskResponseV2 getTask(@PathVariable Long id) {
        return new TaskResponseV2(
                id,
                "Prepare lecture",
                "OPEN"
        );
    }
}

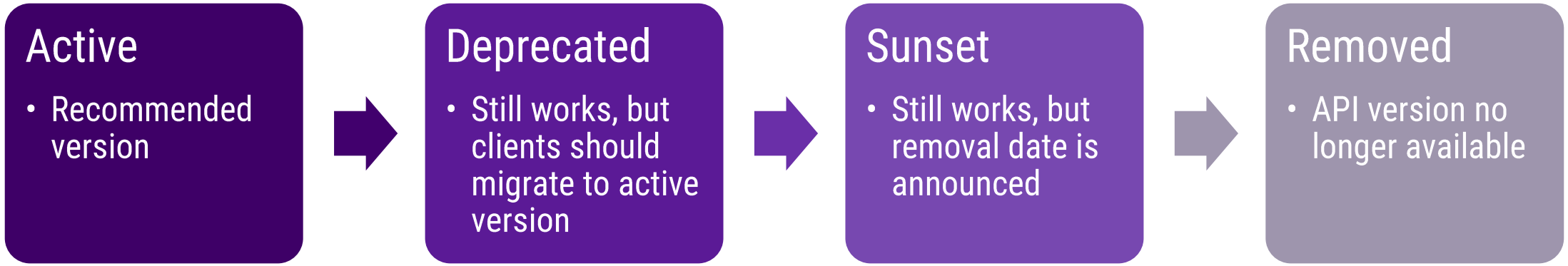
public record TaskResponseV2(
        Long id,
        String title,
        String status
) {}
```

RULES TO VERSION BY

- Do not create a new version for every small change
- Use versioning only when there is a breaking change
- Maintaining multiple versions at the same time is a huge effort!
- Good rule:
  - Add without versioning, remove or redefine with versioning.

DEPRECATION AND SUNSET

- Versioning is not enough
- We also need lifecycle communication
- Generally, we do not want to support an old API forever
- Typical lifecycle includes:



DEPRECATION AND SUNSET

- The important idea is to not break client silently
- You should give them fair warning before a breaking change occurs
- A deprecated endpoint may continue working as intended, but may return additional warning headers
  - E.g.: `Deprecation, Sunset, Link`

HTTP response with warning headers

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Wed, 31 Dec 2026 23:59:59 GMT
Link: </docs/migrate-v1-to-v2>; rel="deprecation"
Content-Type: application/json

{ "foo": "bar" }
```

ADDING WARNING HEADERS: SPRING BOOT

```
@GetMapping("/api/v1/tasks/{id}")
public ResponseEntity<TaskResponseV1> getTaskV1(@PathVariable Long id) {
    TaskResponseV1 body = new TaskResponseV1(
            id,
            "Prepare lecture",
            false
    );

    return ResponseEntity.ok()
            .header("Deprecation", "true")
            .header("Sunset", "Wed, 31 Dec 2026 23:59:59 GMT")
            .header("Link", "</docs/migrate-v1-to-v2>; rel=\"deprecation\"")
            .body(body);
}
```

ADDING WARNING HEADERS: DJANGO NINJA

```python
@api.get("/tasks/{task_id}")
def get_task_v1(request: HttpRequest, task_id: int):
    response = api.create_response(
        request,
        {
            "id": task_id,
            "title": "Prepare lecture",
            "completed": False,
        },
    )
    response["Deprecation"] = "true"
    response["Sunset"] = "Wed, 31 Dec 2026 23:59:59 GMT"
    response["Link"] = '</docs/migrate-v1-to-v2>; rel="deprecation"'
    return response
```

API SECURITY AND HARDENING



API SECURITY RISKS

- The OWASP API Security Top 10 includes:
  1. Broken Object Level Authorization
  2. Broken Authentication
  3. Broken Object Property Level Authorization
  4. Unrestricted Resource Consumption
  5. Broken Function Level Authorization
  6. Unrestricted Access to Sensitive Business Flows
  7. Server Side Request Forgery
  8. Security Misconfiguration
  9. Improper Inventory Management
  10. Unsafe consumption of APIs

BROKEN OBJECT LEVEL AUTHORIZATION

- Dangerous endpoint: `GET /api/tasks/42`
- Client may be authenticated, but task 42 may not belong to them!
  - Developers often forget to properly manage object-level authorization, and over-rely on authentication

```java
public TaskResponse getMyTaskById(Long userId, Long taskId) {
    Task task = taskRepository.findByIdAndOwnerId(taskId, userId)
            .orElseThrow(() -> new ResponseStatusException(
                    HttpStatus.NOT_FOUND, "Task not found"
            ));

    return toResponse(task);
}
```

```python
@api.get("/tasks/{task_id}", response={200: TaskOut, 404: ErrorOut}, auth=jwt_auth)
def get_task(request, task_id: int):
    try:
        task = Task.objects.prefetch_related("tags").get(
            id=task_id,
            owner=request.auth,
        )
    except Task.DoesNotExist:
        return 404, {"error": "Task not found"}

    return task_to_out(task)
```

BROKEN AUTHENTICATION

- Impact:
  - Attackers can gain control of other user's accounts, accessing personal data and performing sensitive operations
- An API is vulnerable if:
  - JWT secret key is exposed or predictable
  - Permits weak passwords
  - Allows users to change email address/password or do sensitive operations without requiring password confirmation
  - Doesn't validate tokens correctly (e.g.: ignores expiration date)
  - Allows attackers to perform brute force attacks without lockout mechanisms

BROKEN OBJECT PROPERTY LEVEL AUTHZ

- Impact:
  - Unauthorized access to private/sensitive object properties may result in unwanted data disclosure
- An API is vulnerable if:
  - Exposes properties of an object that are considered sensitive
  - Allows users to change/add/delete the vallue of a sensitive object property
- Typically associated with returning models directly without using DTOs/Schemas

UNRESTRICTED RESOURCE CONSUMPTION

- APIs often don’t limit client interactions or resource consumption
- This can be exploited by attackers for DoS attacks or to increase operational costs
- An API is vulnerable if at least one of the following limits is set inappropriately:
  - Execution timeouts
  - Rate limit (how many requests per client per unit of time)
  - Maximum allocable memory
  - Maximum number of processes
  - Maximum upload file size
  - Number of records per page to return in a single request-response
  - Third-party service providers' spending limit

RATE LIMITING

- Rate limiting controls how many requests a client can make.
- It helps with:
  - Abuse prevention
  - Brute-force reduction
  - Cost control
  - Fair usage
  - Protecting expensive endpoints
  - Reducing accidental overload
- Enforce a maximum number of requests per user per feature per unit of time

EXPRESS 5 RATE LIMITING

- https://www.npmjs.com/package/express-rate-limit

```
import { rateLimit } from 'express-rate-limit'

const limiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  limit: 10, // Limit each IP to 100 requests per `window` (here, per 15 minutes).
  standardHeaders: 'draft-8',
  legacyHeaders: false, // Disable the `X-RateLimit-*` headers.
  ipv6Subnet: 56,
})

// Apply the rate limiting middleware to all requests.
app.use(limiter)
```

DJANGO NINJA THROTTLING

```
from ninja import NinjaAPI
from ninja.throttling import AnonRateThrottle, AuthRateThrottle

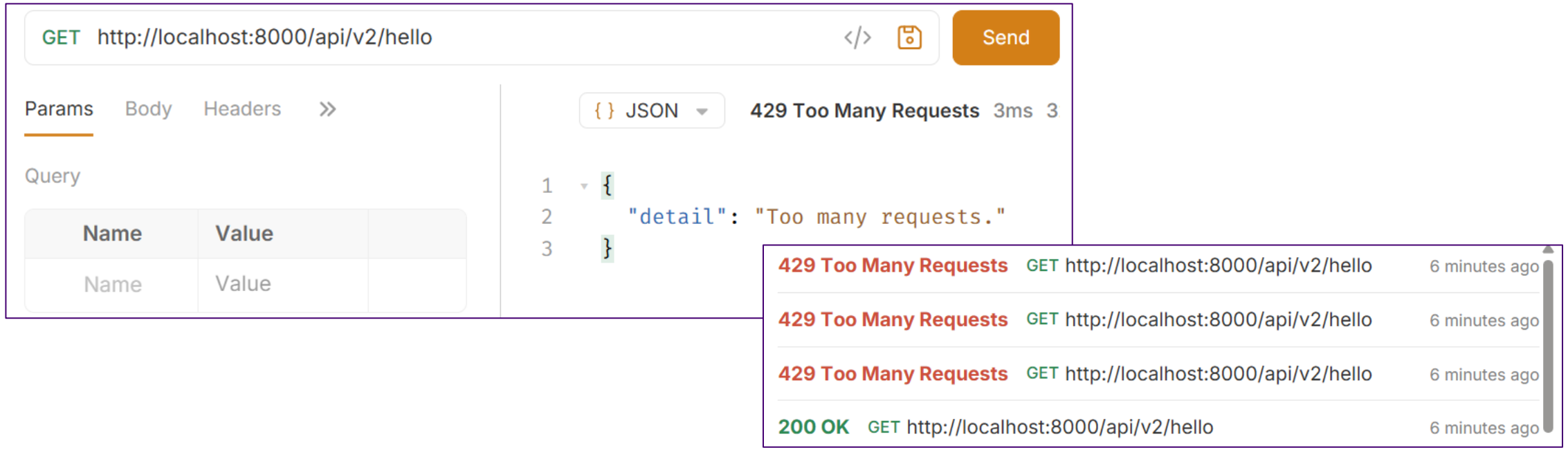
api = NinjaAPI(
    title="Namirial Academy Tasks API",
    version="2.0.0",
    description="REST API for users, tasks, tags, and JWT authentication.",
    throttle=[
        AnonRateThrottle("1/s"),
        AuthRateThrottle("10/s"),
    ]
)
```

- Django Ninja supports global, router-level and method-level throttling: https://django-ninja.dev/guides/throttling/

DJANGO NINJA THROTTLING

- Example with operation-level throttling

```python
@api.post("/login", throttle=[AnonRateThrottle("5/m")])
def login(request, payload: LoginSchema):
    ...
```



SPRING BOOT RATE LIMITING

- If you're interested in implementing a rate limiting mechanism in Spring Boot you may check out libraries like Bucket4j

```
<!-- Bucket4j for java 17+ -->
<dependency>
    <groupId>com.bucket4j</groupId>
    <artifactId>bucket4j_jdk17-core</artifactId>
    <version>8.18.0</version>
</dependency>
```

SPRING BOOT RATE LIMITING

```
@Service
public class LoginRateLimiter {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    public boolean tryConsume(String key) {
        Bucket bucket = buckets.computeIfAbsent(key, this::newBucket);
        return bucket.tryConsume(1);
    }

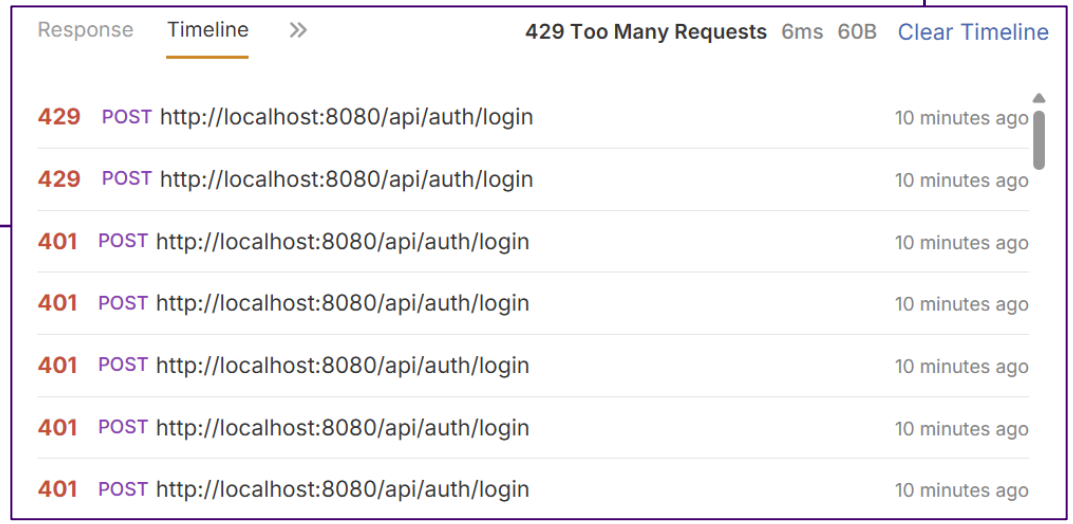
    private Bucket newBucket(String key) {
        Bandwidth limit = Bandwidth.classic(
                5,
                Refill.greedy(5, Duration.ofMinutes(1))
        );
        return Bucket.builder().addLimit(limit).build();
    }
}
```

SPRING BOOT RATE LIMITING

```java
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody AuthRequest request, HttpServletRequest httpRequest) {
    //rate limiting enforcement
    String key = getClientIp(httpRequest) + ":" + request.username();

    if (!loginRateLimiter.tryConsume(key)) {
        return ResponseEntity
                .status(HttpStatus.TOO_MANY_REQUESTS)
                .body(Map.of(
                        "error", "RATE_LIMITED",
                        "message", "Too many login attempts"
                ));
    }

    return ResponseEntity.ok(authService.login(request));
}
```



BROKEN FUNCTION LEVEL AUTHORIZATION

- Authorization flaws that allow attackers to access unauthorized functionality
- Example:
  - Only staff members should be able to `DELETE /api/tags/{id}`, but we implement this check incorrectly (or we don't at all!)

UNRESTRICTED ACCESS TO SENSITIVE FLOWS

- APIs may expose sensitive business flows
- Common examples of sensitive business:
  - Purchasing a product flow - an attacker can buy all the stock of a high-demand item at once and resell for a higher price (scalping)
  - Creating a comment/post flow - an attacker can spam the system
  - Making a reservation - an attacker can reserve all the available time slots and prevent other users from using the system
- Mitigation: Restrict access to sensitive flows…
  - Step 1: Identify sensitive business flows
  - Step 2: Engineering solutions: Device fingerprinting; Human-detection; Detecting non-human usage patterns; Blocking IP addresses…

SERVER-SIDE REQUEST FORGERY

- If an endpoint makes a request to some remote URI provided in requests by client, the URI should be **properly validated**
- Otherwise, attackers can force the API to make requests to unexpected destinations
  - This may result in exposure of sensitive information

```
POST /api/profile/upload_picture

{
  "picture_url": "http://example.com/profile_pic.jpg"
}
```

```
{
  "picture_url": "localhost:8080"
}
```

Based on the response time, the attacker can figure out whether the port is open or not.

SECURITY MISCONFIGURATIONS

- API frameworks typically contain complex configurations, meant to make them more customizable and support many use cases.
- Software and DevOps engineers can miss these configurations, or don't follow security best practices when it comes to configuration, opening the door for different types of attacks.
- Examples:
  - API deployed in debug/dev mode → exposure of stack traces
  - TLS not configured
  - Latest security patches missing

ERROR RESPONSES

- Errors are part of the API Contract
- Mature APIs return predictable and consistent errors

```
{
  "error": "VALIDATION_ERROR",
  "message": "Title must not be blank",
  "details": {
    "field": "title"
  }
}
```

- Avoid leaking internals

```
{
  "error": "NullPointerException at TaskService.java:47"
}
```

IMPROPER INVENTORY MANAGEMENT

- Sometimes, an organization does not clearly know which APIs, endpoints, versions, envs, and data flows are currently exposed.
- This becomes dangerous when old/forgotten APIs remain reachable.

```
/api/v1/admin/users
/api/test/tasks
/api/debug/tokens
/api/v0/legacy-login
```

- Attackers often look for forgotten endpoints because they may have weaker authentication, missing authorization checks, or known vulnerabilities.

UNSAFE CONSUMPTION OF APIs

- Often developers tend to trust data provided by third-party APIs more than client-provided data
- This is dangerous, because external services are outside of our control as well, and should be treated with the same suspicion
- Attackers may compromise trusted third-party APIs
  - Send redirects to malicious services
  - Or send malicious payloads in requests (e.g.: SQL Injections)…

OBSERVABILITY

- In production, we should be able to (automatically) observe the status of the API
  - Is the application running?
  - Is the database reachable?
  - How many requests are failing?
  - Which endpoints are slow?
  - Are users receiving 401, 403, 404, or 500 responses?
  - Are rate limits being triggered?
  - Are old API versions still used?
- Without **observability**, production problems become guessing

OBSERVABILITY IN SPRING BOOT

- Add the Actuator dependency to the pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

- Add the following line to application.properties to expose info and metrics data

```
#Actuator config
management.endpoints.web.exposure.include=*
```

OBSERVABILITY IN SPRING BOOT

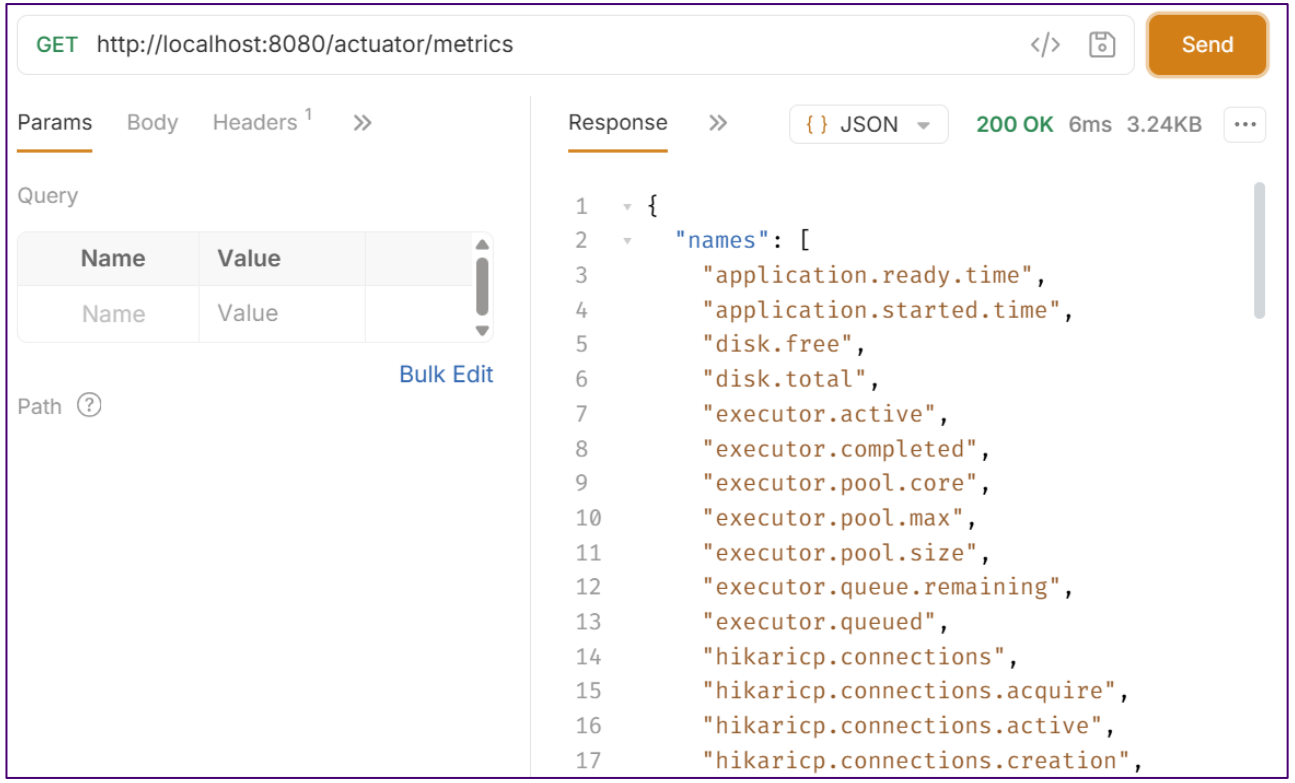
- You can make a request to /actuator/health to get status details



- Possible status values: UP, DOWN, OUT_OF_SERVICE; UNKNOWN

OBSERVABILITY IN SPRING BOOT

- You may also get more detailed metrics using the `actuator/metrics` endpoint
- The GET returns a list of available metrics
- You need to make follow up get requests to `/actuator/metrics/{name}` to get the corresponding values



OBSERVABILITY IN SPRING BOOT

- For example, a request to `/actuator/metrics/system.cpu.usage` may return

Response Headers 12 Timeline Tests { } JSON 200 OK 8ms 198B


```
{
  "availableTags": [],
  "description": "The \"recent cpu usage\" of the system the application is running in",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 0.13948267002189407
    }
  ],
  "name": "system.cpu.usage"
}
```

MINIMAL OBSERVABILITY IN DJANGO NINJA

```python
@api.get("/health", auth=None)
def health(request):
    with connection.cursor() as cursor:
        cursor.execute("SELECT 1")

    return {
        "status": "ok",
        "database": "ok"
    }
```

LOGGING

- Useful logs:
  - Authentication failures
  - Authorization failures
  - Rate limit violations
  - Unexpected server errors
  - Deprecated API usage
  - Admin actions
  - Data-changing operations
  - External service failures
- Avoid logging:
  - Passwords
  - Tokens
  - Full Authorization headers
  - Sensitive personal data
  - Full request bodies by default
  - **Logs are operational tools, but they can also become security risks.**

REFERENCES (1/2)



- **The Little Book on REST Services**
  By Kenneth Lange
  Freely available at https://www.kennethlange.com/books/The-Little-Book-on-REST-Services.pdf
- **API design guide**
  Google Cloud
  https://cloud.google.com/apis/design
  Relevant parts: Resource-oriented design, Resource names.
- **REST API Checklist**
  By Kenneth Lange
  https://kennethlange.com/rest-api-checklist/
- **REST API Design Rulebook: Designing Consistent RESTful Web Service Interfaces**
  By Mark Massé

REFERENCES (2/2)

- **OpenAPI**
  Official Website
  https://www.openapis.org/
  https://learn.openapis.org/ (Recommended reads from this link: Getting started, Introduction)
  https://spec.openapis.org/oas/v3.1.0 (Full specification for OpenAPI 3.1.0)
  ⚠️ For the Web Technologies course, you need to know what the OpenAPI specification standard is, why it has been introduced, and what it can be used for. You are **not** required to learn how to write OpenAPI specification files from scratch.
- **API Documentation: The Secret to a Great API Developer Experience**
  Ebook by Smartbear Software
  Available at https://swagger.io/resources/ebooks/api-documentation-the-secret-to-a-great-api-developer-experience and archived here.
  ⚠️ Interesting read. **Not** required for the Web Technologies course, but it can be useful to better understand the challenges in designing and documenting an API.

EXAMPLES WITH DIFFERENT FRAMEWORKS

- **To-do List REST API using JakartaEE (Java)**
  By Luigi Libero Lucio Starace
  Includes source code, Dockerfile and instructions.
  https://github.com/luistar/jakartaee-rest-example
- **Pizza Shop REST API using Spring (Java)**
  By Luigi Libero Lucio Starace
  https://github.com/luistar/rest-service-pizzashop
- **Quiz REST API using FastAPI (Python)**
  By Márcio Lemos
  https://github.com/marciovrl/fastapi

⚠️ You are **not** required to learn JakartaEE, Spring, or FastAPI for the Web Technologies course. It might be educationally worthwhile to take a look at some web frameworks other than Express. JakartaEE and Spring, for example, use an annotation-based approach. So, feel free to look at the code or try to run the above linked REST API, but be aware that it's not required for this course.